

DrawPanel måste generaliseras (fixat nu)

För MVC:

Flytta TimeListener till application (inte relevant till user input)

Design patterns:

Observer pattern används i View för att kapa dependency mellan view och CarController

Frågor:

1. se implementation
2. a) CarView och CarController har cirkulärt beroende
  - Bryter mot designfilosofier för MVC och leder till mindre tydlig gränsdragning
  - CarController innehåller funktionalitet som är orelaterad till user input
  - Kan göras tunnare om detta flyttas någon annanstans
  - DrawPanel håller koll på punkter, inte jobb för en view.
  - DrawPanel blir för smart

b) Cirkulärt beroende mellan CarView och CarController bröts med vår lösning. DrawPanel blev mer generell, men höll fortfarande koll på punkter när det inte behövdes. TimeListener lämnades kvar i CarController i vår design.

3. Observer pattern:
  - Används GameGraphics (view) för att bryta dependency mellan view och carcontroller. dessa beror istället på ett interface ControlMedium som innehåller relevanta metoder.

Vi tror inte att observer pattern hade lönats att använda någon annan stans, då inga andra dependencies anses behöva brytas.

Factory Method pattern:

Just nu används inte detta pattern någonstans i koden.

Vi hade dock möjligen sett fördelar av att använda detta, nämligen genom att skapa en factory för att producera olika sorters bilar. Detta hade gjort det lättare att skala upp vårt program för fler sorters bilar i framtiden.

State pattern:

State pattern används just nu inte i vår kod.

Vi hade dragit nytta av detta pattern för t.ex start och stopp-funktion för bilarna och lastet på Scanian och CarTransport. Detta hade gjort model lättare att hantera och intuitiv för användaren.

Vi anser dock just nu att dessa implementationer hade varit onödiga, då vi har väldigt få sådana states i vår kod, och de är alla väldigt enkla.

Composite pattern:

Composite pattern används något oavsiktligt i vårt program i view, där DrawPanel (som hanterar själva spelet) och CarView (som hanterar spelets interface) slås ihop till en fungerande view i GameGraphics.

Composite pattern hade även kunnat användas i t:ex vehicle i model för att bryta upp bilen i flera komponenter (engine, wheels, chassi etc.) detta hade gett oss en mer modulär design, och gjort det lättare att implementera andra typer av fordon i framtiden, men vi är osäkra på om detta är fullt nödvändigt.

Uppgift 5:

Här använder vi oss dels av Observer pattern för att behålla MVC modellen och inte skapa dependency mellan CarController och DrawPanel. Samt att vi använder Factory Pattern för att generera bilar.